OSSP

ABSTRACT

Roll No: -2520080021,2520080017,2520090074

Project Abstract: Linux Kernel Monitor Section-13

1. Problem Statement

Monitoring the activities and resource usage of the Linux operating system is important for understanding system performance and identifying resource-intensive processes. However, manually checking information from different Linux utilities and files such as /proc can be inconvenient. This project aims to develop a **Linux Kernel Monitor** that collects and displays important system and process information in a centralized interface.

2. Objectives

- To monitor the current state of the Linux system.
- To display information about running processes and their PIDs.
- To monitor CPU and memory utilization.
- To display system uptime and resource usage.
- To provide basic process monitoring and control features.
- To understand how Linux exposes kernel-level information to user-space programs.

3. Proposed Approach / Methodology

The proposed system will be implemented as a **user-space Linux monitoring application** that retrieves information exposed by the Linux kernel. The application will read relevant information from the /proc filesystem and use POSIX/Linux system calls to obtain process and system details. It will periodically refresh the collected information and display CPU usage, memory usage, uptime, and running processes. Basic process-control operations such as terminating or sending signals to a selected process can also be provided. The monitor will present the information through a command-line interface for easy interaction.

4. Linux/POSIX Technologies or System Calls to Be Used

- **C Programming** – primary implementation language.
- **Linux /proc filesystem** – to obtain kernel and process information.
- open() – to open system and process information files.
- read() – to read information from /proc.
- close() – to close file descriptors.
- opendir() / readdir() – to explore process directories in /proc.
- getpid() – to obtain the current process ID.
- kill() – to send signals to processes for basic process control.
- sleep() – to periodically refresh monitoring information.
- sysconf() – to obtain system configuration information such as CPU count.

5. Expected Outcome

The project is expected to produce a functional **Linux Kernel Monitoring System** capable of displaying real-time or periodically refreshed information about CPU usage, memory utilization, uptime, and running processes. It will also provide basic process-control functionality where appropriate. The project will demonstrate practical understanding of **Linux kernel interfaces, the /proc filesystem, system calls, process management, and resource monitoring**.